Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Qual è il compito del **DNS**?

- **A** Tradurre un nome di dominio (es. `www.unina.it`) nell'indirizzo IP del server.
- B Trasformare una URL relativa in una URL assoluta.
- C Instradare i pacchetti IP fino al server di destinazione.
- D Memorizzare in cache le pagine visitate di recente.
- E Assegnare un indirizzo IP al computer quando si collega alla rete.
- F Cifrare le richieste HTTP prima che lascino il browser.

Risposta: A. Il DNS è una «rubrica»: dato un nome restituisce un indirizzo. Una volta ottenuto l'indirizzo IP, il browser apre la connessione e invia la richiesta HTTP; il DNS non interviene nel trasferimento dei dati. *(Dispensa: cap. 1, «Cosa succede quando si apre una pagina»)*

Es. **2.** **(2 punti)** Una pagina scritta in UTF-8 non dichiara la codifica dei caratteri e il server non la indica nella risposta. Che cosa può succedere?

- A Il server risponde con il codice di stato `415 Unsupported Media Type`.
- B Gli elementi della pagina vengono visualizzati senza alcuno stile.
- C Il browser mostra il codice sorgente della pagina invece del contenuto.
- D Il browser rifiuta di visualizzare la pagina e mostra un errore.
- E Le immagini non vengono caricate perché il percorso non è valido.
- **F** Le lettere accentate possono comparire come sequenze di simboli senza senso.

Risposta: F. Senza `<meta charset="utf-8">` (o l'intestazione `Content-Type` con il charset) il browser deve indovinare la codifica: se sbaglia, i caratteri non ASCII come «è» diventano sequenze di simboli (*mojibake*). *(Dispensa: cap. 2, «L'elemento head»)*

Es. **3.** **(2 punti)** Un selettore con **undici** classi e un selettore con **un** id selezionano lo stesso elemento e impostano proprietà in conflitto. Quale prevale?

- **A** Quello con l'id, perché la specificità si confronta per componenti.
- B Quello con l'id, ma solo se è marcato `!important`.
- C Il più recente nell'ordine del foglio di stile, perché la specificità è pari.
- D Quello con le undici classi, perché 11 è maggiore di 1.
- E Quello con le undici classi, ma solo se è scritto dopo.
- F Nessuno dei due: in caso di conflitto si applica il valore predefinito.

Risposta: A. La specificità è una terna (id, classi, tipi) confrontata da sinistra: (1, 0, 0) batte (0, 11, 0). Non è un numero a tre cifre in cui le classi «riportano» negli id. *(Dispensa: cap. 3, «Specificità»)*

SOLUZIONI

Es. **4.** **(2 punti)** Si consideri il seguente frammento di codice JavaScript (si assuma la presenza di `"use strict";`).

```
1  console.log(x);
2  if (true) {
3    console.log(x);
4    let x;
5    console.log(x); }
```

Con riferimento al ciclo di vita di una variabile dichiarata con `let`, indicare quali delle seguenti affermazioni sono **vere**.

- [x] A — In Linea 3, `x` è dichiarata ma non inizializzata.
- [ ] B — In Linea 3, `x` è inizializzata a `undefined`.
- [x] C — In Linea 5, `x` ha valore `undefined`.
- [ ] D — Il codice esegue senza sollevare errori.
- [ ] E — In Linea 5, `x` ha valore `null`.
- [ ] F — In Linea 1, `x` è inizializzata ma non dichiarata.

Risposte: A, C. Una variabile `let` viene dichiarata (sollevata) all'inizio del suo blocco, ma resta non inizializzata fino alla riga della dichiarazione: dalla linea 2 alla 4 è nella *temporal dead zone*, e usarla solleva un `ReferenceError`. Con `let x;` viene inizializzata a `undefined`. In Linea 1, fuori dal blocco, `x` non esiste affatto: l'esecuzione si ferma proprio lì con un `ReferenceError`. *(Dispensa: cap. 5, «Il ciclo di vita di una variabile») — Fonte: prima prova intercorso del 23 aprile 2024.*

Es. **5.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  async function f() { return 1; }
2  const r = f();
3  console.log(typeof r, r instanceof Promise);
4  r.then((v) => console.log(v));
5  console.log("fine");
```

- [ ] A — `number false`, poi viene sollevato un `TypeError`.
- [ ] B — `object true`, `1`, `fine`.
- [ ] C — `number false`, `fine`, `1`.
- [x] D — `object true`, `fine`, `1`.
- [ ] E — `number false`, `1`, `fine`.
- [ ] F — `function false`, `fine`, `1`.

Risposta: D. Una funzione `async` restituisce **sempre** una Promise: il valore restituito (qui 1) viene avvolto in una Promise risolta. Il gestore registrato con `then` viene eseguito dopo la fine del codice sincrono. *(Dispensa: cap. 8, «async»)*

Es. **6.** **(2 punti)** Secondo la specifica CGI, come arriva al programma esterno il corpo di una richiesta `POST`?

- [ ] A — In un file temporaneo il cui nome è passato al programma.
- [ ] B — Tramite un cookie impostato dal server.
- [ ] C — Come argomenti sulla riga di comando del programma.
- [ ] D — È ciò che il programma scrive sullo **standard output**.
- [x] E — Sullo **standard input** del programma.
- [ ] F — Come **variabili d'ambiente**.

Risposta: E. La specifica CGI stabilisce come server e programma comunicano: header e query string arrivano come variabili d'ambiente, il corpo della richiesta sullo standard input; ciò che il programma scrive sullo standard output (header, riga vuota, corpo) diventa la risposta. *(Dispensa: cap. 10, «CGI: Common Gateway Interface»)*

SOLUZIONI

Es. **7.** **(2 punti)** Si consideri il seguente server. Quando viene eseguita la funzione passata a `createServer`?

```
1  import http from 'http';
2
3  let server = http.createServer(function (request, response) {
4    response.writeHead(200, { "Content-Type": "text/html" });
5    response.end(`<p>Sono le ${new Date().toTimeString()}</p>`);
6  });
7  server.listen(3000);
```

- [A] Ogni volta che arriva una richiesta: è una callback, invocata dall'event loop.
- [B] Una sola volta, all'avvio del server.
- [C] Solo per la prima richiesta: le successive ricevono la stessa risposta.
- [D] Quando viene chiamata `listen`, per ciascuna porta.
- [E] Una volta al secondo, finché il server resta acceso.
- [F] Solo per le richieste `POST`.

Risposta: A. `createServer` registra la funzione come gestore delle richieste: l'event loop la invoca a ogni richiesta, e la pagina viene costruita ogni volta (qui con l'ora corrente). Non c'è alcun file HTML sul disco. *(Dispensa: cap. 11, «Una prima applicazione web con Node.js»)*

Es. **8.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono tre richieste `GET /contatore`: la prima senza cookie, la seconda con l'header `Cookie: count=41`, la terza con l'header `Cookie: count=abc`?

```
1  import express from "express";
2  import cookieParser from "cookie-parser";
3
4  const app = express();
5  app.use(cookieParser());
6
7  app.get("/contatore", (req, res) => {
8    const n = req.cookies.count ? parseInt(req.cookies.count) + 1 : 1;
9    res.cookie("count", n, { maxAge: 10 * 60 * 1000 });
10   res.send(`Visita numero ${n}`);
11 });
12
13 app.listen(3000);
```

- [A] `200 Visita numero 1`, `200 Visita numero 42`, `200 Visita numero 1`.
- [B] `200 Visita numero 1`, `200 Visita numero 42`, `200 Visita numero NaN`.
- [C] `200 Visita numero 1`, `200 Visita numero 1`, `200 Visita numero 1`.
- [D] `200 Visita numero 1`, `200 Visita numero 2`, `200 Visita numero 3`.
- [E] `200 Visita numero 1`, `200 Visita numero 42`, `400 Bad Request`.
- [F] `200 Visita numero 1`, `200 Visita numero 411`, `200 Visita numero abc1`.

Risposta: B. `cookie-parser` mette i cookie della richiesta in `req.cookies`, come stringhe. Il contatore sta nel client, che può inviare il valore che vuole: il server crede a `41`, e con `abc` `parseInt` dà `NaN`. Con `express-session` il contatore starebbe invece sul server, e il cookie conterrebbe solo l'identificatore firmato della sessione. *(Dispensa: cap. 15, «cookie-parser»)*

Es. **9.** **(5 punti)** Si descrivano i passi che il browser compie dal clic su un link `https://` alla visualizzazione della risposta, indicando quali si possono evitare nelle visite successive.

Traccia della risposta.

- Analisi della URL (schema, host, porta, percorso); risoluzione DNS del nome in indirizzo IP.
- Apertura della connessione (TCP o QUIC) verso IP e porta; negoziazione TLS (certificato, chiavi di sessione).
- Invio della richiesta (metodo, destinazione, header, corpo); attesa ed elaborazione del server; ricezione della risposta (stato, header, corpo).
- Visite successive: DNS in cache, connessione persistente già aperta (niente connessione né TLS), risorse in cache verificate con `ETag` e `304`.

(Dispensa: cap. 9, «Dal clic alla pagina: il viaggio completo»)

SOLUZIONI

Es. **10.** **(5 punti)** Si spieghi il problema del tracciamento della sessione in HTTP e si confrontino l'approccio che memorizza lo stato nei cookie e quello basato sulle sessioni web.

Traccia della risposta.

- HTTP è *stateless*: per riconoscere un utente fra più richieste (es. dopo il login) serve uno stato mantenuto dall'applicazione.
- Stato nei cookie (`Set-Cookie: auth=true`, `username=...`): semplice, ma i cookie sono controllati dal client e si manomettono facilmente; limiti di dimensione e numero. I cookie firmati riconoscono la manomissione ma lasciano i dati al client.
- Sessioni: i dati restano sul server, in una struttura chiave-valore; il client riceve solo un identificatore opaco (di solito un cookie) che il server usa per ritrovarli.
- L'identificatore deve essere imprevedibile (generatore crittograficamente sicuro): chi lo conosce è il proprietario della sessione.

(Dispensa: cap. 13, «Le sessioni web»)

Es. **11.** **(5 punti)** Che cos'è un ORM e quali problemi risolve? Si descriva, con Sequelize, come si definiscono i modelli e le associazioni, e si discutano pro e contro di questo approccio.

Traccia della risposta.

- Gli stessi dati hanno due rappresentazioni (oggetti nello heap, righe nelle tabelle) da tenere allineate: un ORM lo fa con modelli dichiarativi, schema generato, CRUD e interrogazioni come metodi.
- Sequelize: `database.define(...)` o classi che estendono `Model` con `init`; tipi e vincoli (`allowNull`, `primaryKey`, `autoIncrement`); `sync()` allinea lo schema.
- Associazioni: `hasOne`, `belongsTo`, `hasMany`, `belongsToMany` (con `through`); nell'uno-a-molti la chiave esterna (es. `UserId`) sta nella tabella dei «molti».
- Pro: astrazione, produttività, portabilità, leggibilità. Contro: curva di apprendimento, operazioni complesse più contorte, *vendor lock-in*, prestazioni.

(Dispensa: cap. 16, «Object-Relational Mapping»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	A	1	DNS e caricamento di una pagina
2	F	2	l'elemento head
3	A	3	cascata e specificità
4	A, C	5	variabili, scope e hoisting
5	D	8	async e await
6	E	10	CGI
7	A	11	un server HTTP con Node.js
8	B	15	sessioni e cookie
9	aperta	9	HTTPS
10	aperta	13	sessioni web
11	aperta	16	ORM e Sequelize

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI